Clé ssh / github et container (1)

La clé SSH permet une connexion sécurisée à GitHub sans avoir à entrer votre mot de passe à chaque fois.
La clé SSH utilise le port 22. La clé SSH regroupe la clé publique qui peut être partagé (a github notamment) et la clé privée qu'il faut surtout pas
Adresse des clé ssh sur le pc C:\Users\l14ad\.ssh

Étape 1 : Génération de la clé SSH

Sur votre machine hôte (pas dans le container) :

bash

```
# Générer une paire de clés SSH
ssh-keygen -t ed25519 -C "votre.email@example.com"

# Quand demandé :
# - Emplacement : appuyez sur Entrée (utilise ~/.ssh/id_ed25519 par défaut)
# - Passphrase : optionnel mais recommandé pour plus de sécurité
```

Cela crée deux fichiers :

- `~/.ssh/id_ed25519` (nom par défaut si non nommé) : **clé privée** (ne JAMAIS partager)
- `~/.ssh/id_ed25519.pub` (nom par défaut si non nommé) : **clé publique** (à ajouter sur GitHub)

Vérifier les permissions (important pour la sécurité) :

bash

```
chmod 600 ~/.ssh/id_ed25519
chmod 644 ~/.ssh/id_ed25519.pub
```

Étape 2 : Ajouter la clé publique à GitHub

Copier le contenu de votre clé publique :

bash

```
cat ~/.ssh/id_ed25519.pub
# Copier tout le contenu affiché
```

1. Sur GitHub :
    - Aller dans **Settings** (icône profil en haut à droite)
    - Cliquer sur **SSH and GPG keys** dans le menu de gauche
    - Cliquer sur **New SSH key**
    - Titre : donner un nom descriptif (ex: "Mon PC - Projet Apache")
    - Key : coller le contenu de votre clé publique
    - Cliquer sur **Add SSH key**

Étape 3 : Configuration Git locale

Avec PowerShell :

powershell

```
# Configurer votre identité
git config --global user.name "Votre Nom"
git config --global user.email "votre.email@example.com"

# Configurer la fin de ligne (important pour Windows)
git config --global core.autocrlf true

# Vérifier la configuration
git config --list
```

Étape 4 : Créer un fichier config

Créer un fichier sans extension et indiqué votre nom de clé privée en rouge. Si vous avez plusieurs clés, ajouter en un autre à la suite dans le même fichier.

```
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/github
    IdentitiesOnly yes
```

- Créer un dossier ssh-key et copier les clés publique et privée.

Étape 6 : vérifier la connexion avec github

Taper la commande dans le cmd :

ssh -vT git@github.com

ou pour ne pas avoir tous les détails

ssh -T git@github.com

- S'il vous dit que l'accès est denied c'est que vous n'avez pas fait correctement votre fichier config.
- S'il vous dit que l'accès est pas assez sécurisé, il faut aller dans les propriétés du fichier de la clé et supprimer les utilisateurs qui y ont accès pour en avoir qu'un seul.

Étape 7 : Créer votre repository GitHub et cloner localement

Si le repository n'existe pas encore :

1. Sur GitHub, créer un nouveau repository
2. **Ne pas** initialiser avec README, .gitignore ou licence (vous le ferez localement)

Étape 8a : Connecter Virtual studio code avec github

- Télécharger l'extension Github pull request et le lancer.
-

Étape 8b : Créer le site

- Ouvrir Visual studio code et télécharger l'extension suivante: Github pull request.
- Une fois installé, aller à la page d'accueil (welcome) et cliquer sur " clone git repository..."
- Sélectionner en haut au niveau de la barre de recherche "clone from git"
- Puis sélectionner le repository souhaité.
- Dans explorer, vous aurez un menu défilant avec votre repo.
- Cliquer dessus et crée un new file. (index.html par exemple)
- Apres avoir fait votre site, vous pouvez commit et syncro avec git.
- Vérifier que git se met a jour.

Étape 9 : Configuration Docker Compose + dockerfile + entrypoint.sh

1. Le docker compose

```
version: '3.8'
services:
    apache:
        build: .
        container_name: apache_server
        ports:
```

```
            - "91:80"
            - "6443:443"
            - "2200:22"
        volumes:
            - ./html:/usr/local/apache2/htdocs
            - ./conf:/usr/local/apache2/conf/extra
        environment:
            - TZ=UTC
        restart: unless-stopped
        networks:
            - apache_network

networks:
    apache_network:
        driver: bridge
```

2. Le docker file

Ce Dockerfile crée un **container multi-services combinant un serveur web Apache avec un déploiement automatisé via Git**. C'est une solution "tout-en-un" pour héberger un site web qui se synchronise automatiquement avec un repository GitHub.

```
FROM httpd:latest

RUN apt update && apt ins
tall -y nano openssh-serv
er git && apt clean && rm
-rf /var/lib/apt/lists/*

RUN mkdir -p /var/run/ssh
d && \
    sed -i 's/#PermitRoot
Login prohibit-password/P
ermitRootLogin yes/' /et
c/ssh/sshd_config && \
```

Ligne	Instruction
1	`FROM httpd:latest`
3	`RUN apt update && apt install...`

```
    sed -i 's/#PubkeyAuth
entication yes/PubkeyAuth
entication yes/' /etc/ss
h/sshd_config

RUN echo 'root:Password1'
| chpasswd
RUN mkdir -p /root/.ssh &
& chmod 700 /root/.ssh
COPY ssh-key/github.pub /
root/.ssh/id_ed25519.pub
COPY ssh-key/github /roo
t/.ssh/id_ed25519
RUN chmod 600 /root/.ssh/
id_ed25519 && \
    chmod 644 /root/.ssh/
id_ed25519.pub && \
    cat /root/.ssh/id_ed2
5519.pub >> /root/.ssh/au
thorized_keys && \
    chmod 600 /root/.ssh/
authorized_keys

RUN git config --global u
ser.name "duckzla" && \
    git config --global u
ser.email "l14.adm@gmail.
com"

RUN echo "ServerName loca
lhost" >> /usr/local/apac
he2/conf/httpd.conf

EXPOSE 80 22 443

COPY entrypoint.sh /entry
point.sh
```

Ligne	Instruction
5-7	`RUN mkdir -p /var/run/sshd...`
9	`RUN echo 'root:Password1'...`
10	`RUN mkdir -p /root/.ssh...`
11	`COPY ssh-key/github.pub...`
12	`COPY ssh-key/github...`
13-16	`RUN chmod 600...`

```
RUN sed -i 's/\r$//' /entrypoint.sh

RUN chmod +x /entrypoint.sh

ENTRYPOINT ["/entrypoint.sh"]
```

Ligne	Instruction
18-19	`RUN git config --global...`
21	`RUN echo "ServerName localhost"...`
23	`EXPOSE 80 22 443`
25	`COPY entrypoint.sh...`
27	`RUN sed -i 's/\r$//'...`
29	`RUN chmod +x /entrypoint.sh`
31	`ENTRYPOINT ["/entrypoint.sh"]`

3. Le entrypoint.sh

Utilité du script entrypoint.sh: Ce script **point d'entrée** est exécuté automatiquement au démarrage du container. Il orchestre trois fonctions principales :

1. **Démarrer le serveur SSH** pour permettre l'accès distant
2. **Synchroniser le code source** depuis GitHub vers le répertoire web d'Apache
3. **Lancer Apache** en mode foreground pour maintenir le container actif

```
#!/bin/bash

# Démarrer SSH
service ssh start

# Configuration
REPO_URL="git@github.com:duckzla/site1.git"
WEB_DIR="/usr/local/apache2/htdocs"
BRANCH="main"
# ou "master" selon votre repo

# Cloner ou mettre à jour le repository
if [ -d "$WEB_DIR/.git" ]; then
    echo "Repository déjà cloné, mise à jour..."
    cd $WEB_DIR
```

Ligne	Commande
1	`#!/bin/bash`
3	(commentaire)
4	`service ssh start`
6	(commentaire)
7	`REPO_URL="git@github.com:..."`
8	`WEB_DIR="/usr/local/apache2/htdocs"`
9	`BRANCH="main"`
11	(commentaire)

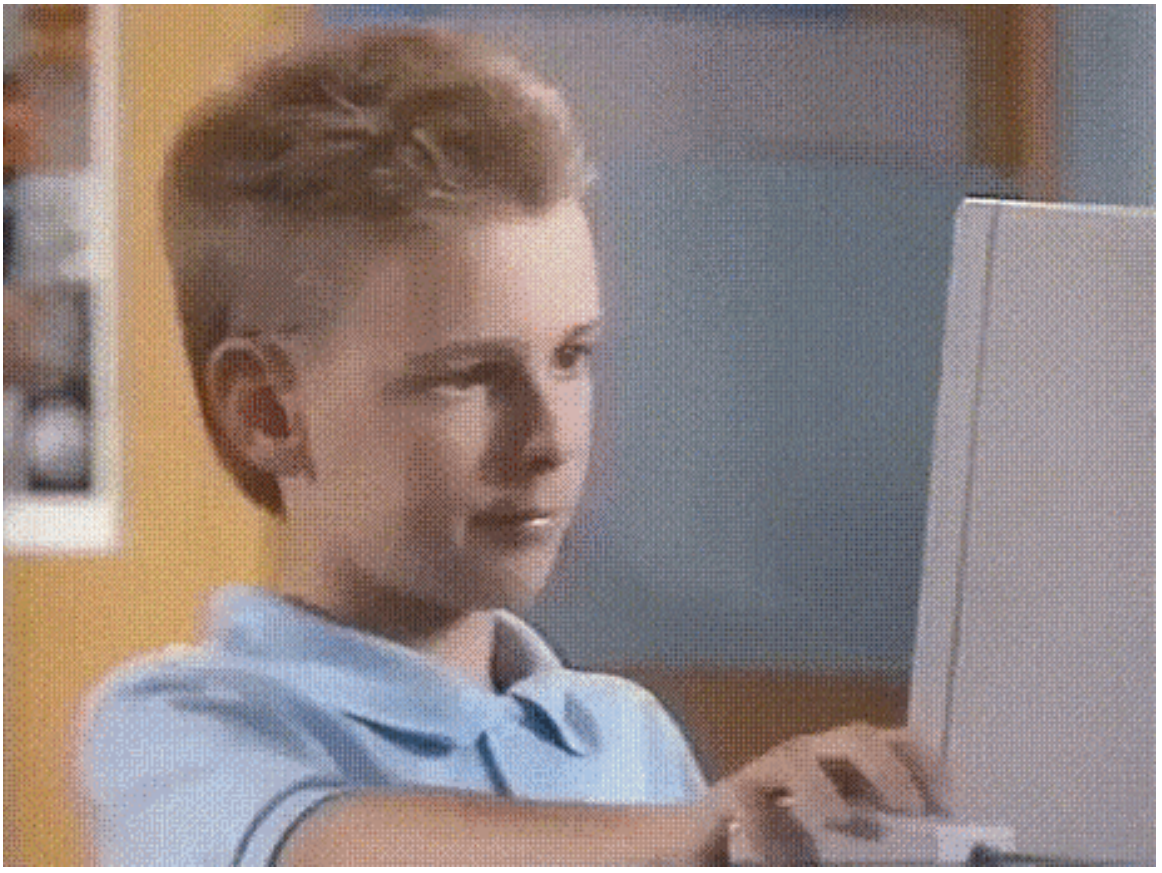
```
    git pull o
rigin $BRANCH
else
    echo "Clon
age du reposit
ory..."
    rm -rf $WE
B_DIR/*
    git clone
-b $BRANCH $RE
PO_URL $WEB_DI
R
fi

# Afficher le
statut
echo "Reposito
ry cloné/mis à
jour avec succ
ès!"
cd $WEB_DIR &&
git log -1 --o
neline

# Démarrer Apa
che
httpd-foregrou
nd
```

Ligne	Commande
12	`if [ -d "$WEB_DIR/.git" ]`
13	`echo "Repository déjà cloné..."`
14	`cd $WEB_DIR`
15	`git pull origin $BRANCH`
16	`else`
17	`echo "Clonage du repository..."`
18	`rm -rf $WEB_DIR/*`
19	`git clone -b $BRANCH $REPO_URL $WEB_DIR`
20	`fi`
22	(commentaire)

Ligne	Commande
23	`echo "Repository cloné/mis à jour..."`
24	`cd $WEB_DIR && git log -1 --oneline`
26	(commentaire)
27	`httpd-foreground`

Étape 10 : Lancé le docker compose.

Il suffit de lancer le docker compose et tout devrait fonctionner !



Annexes:

Bonus 1: Supprimer une clé SSH

Il suffit de supprimer la clé publique/ privée et les fichiers hosts. Le fichier config ne doit pas être supprimé.

Bonus 2: entrer dans le container pour faire réaliser des commandes

Commande permettant d’exécuter des commandes dans les containers:

docker exec -it nomducontainer sh / pour sortir du container taper exit

On peut retaper la commande suivante pour vérifier la pont avec github:

ssh -vT git@github.com

Se connecter en ssh a un container (2200 c'est le port de mon container):

ssh -p 2200 root@localhost